

Making Content Defined Chunking into a First-Class Citizen

An architectural overview

Agenda

- The REv2 Data Model in Brief
- The Large Blob Problem
- Content Defined Chunking
- Changes to Buildbarn
- Results and Measurements

REv2 Data Model in Brief

Action

```
platform:
 os: linux
 arch: x86_64
command:
 args: gcc main.c -o main
 envs
   PATH: /usr/bin
input: 5309140e-625
```

Input

```
subDirectories:
 libs/: cb5c374a-314
files:
 main.c: 7672f03c-1027
 main.h: 8fab43d6-1337
```

Output

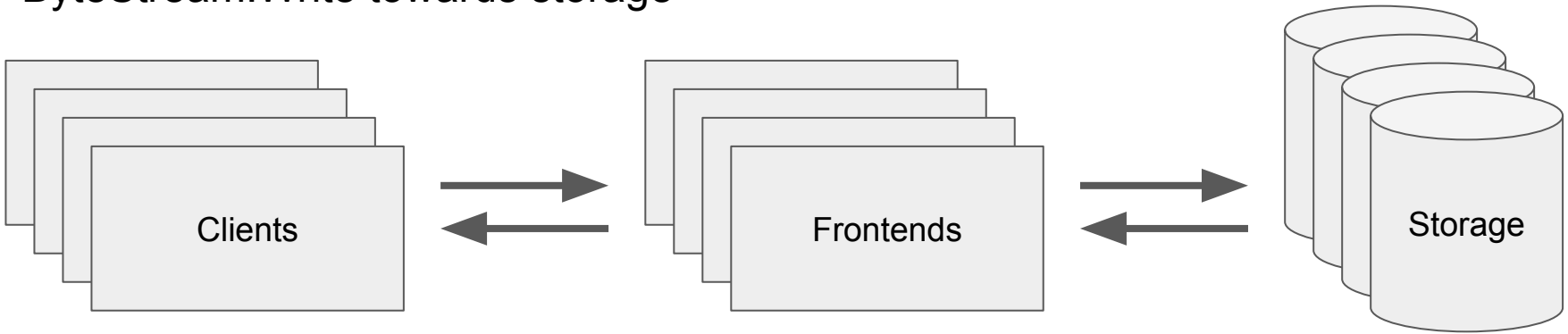
```
subDirectories:
  <empty>
files:
  main.o: 7593b459-36679
```



The Large Blob Problem

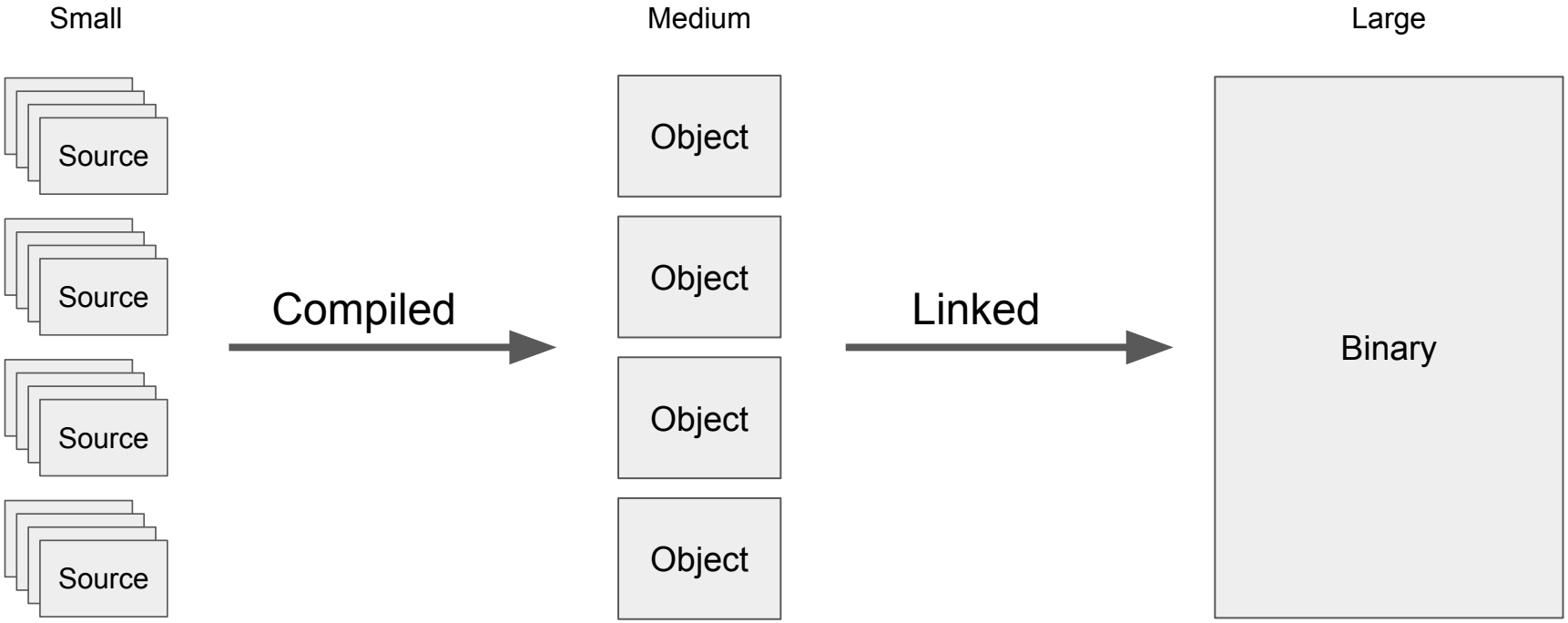
The Large Blob Problem

ByteStream.Write towards storage

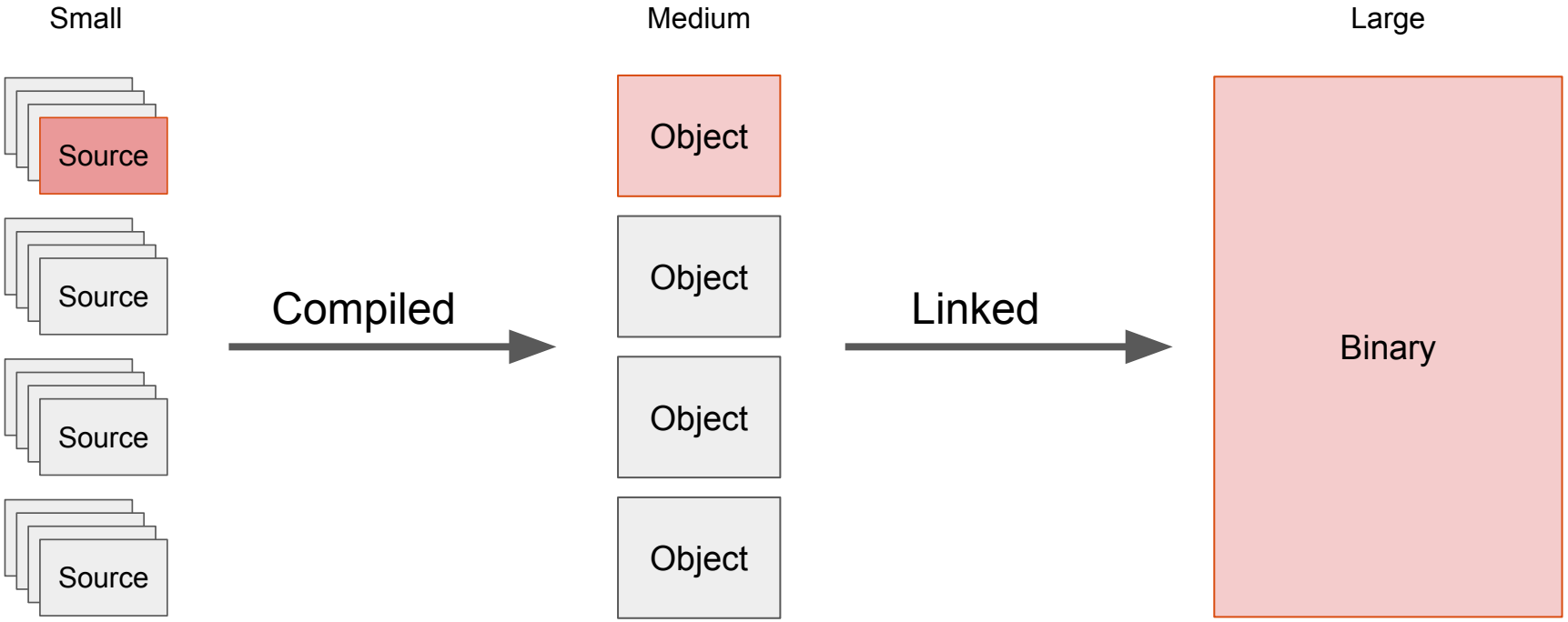


ByteStream.Read from storage

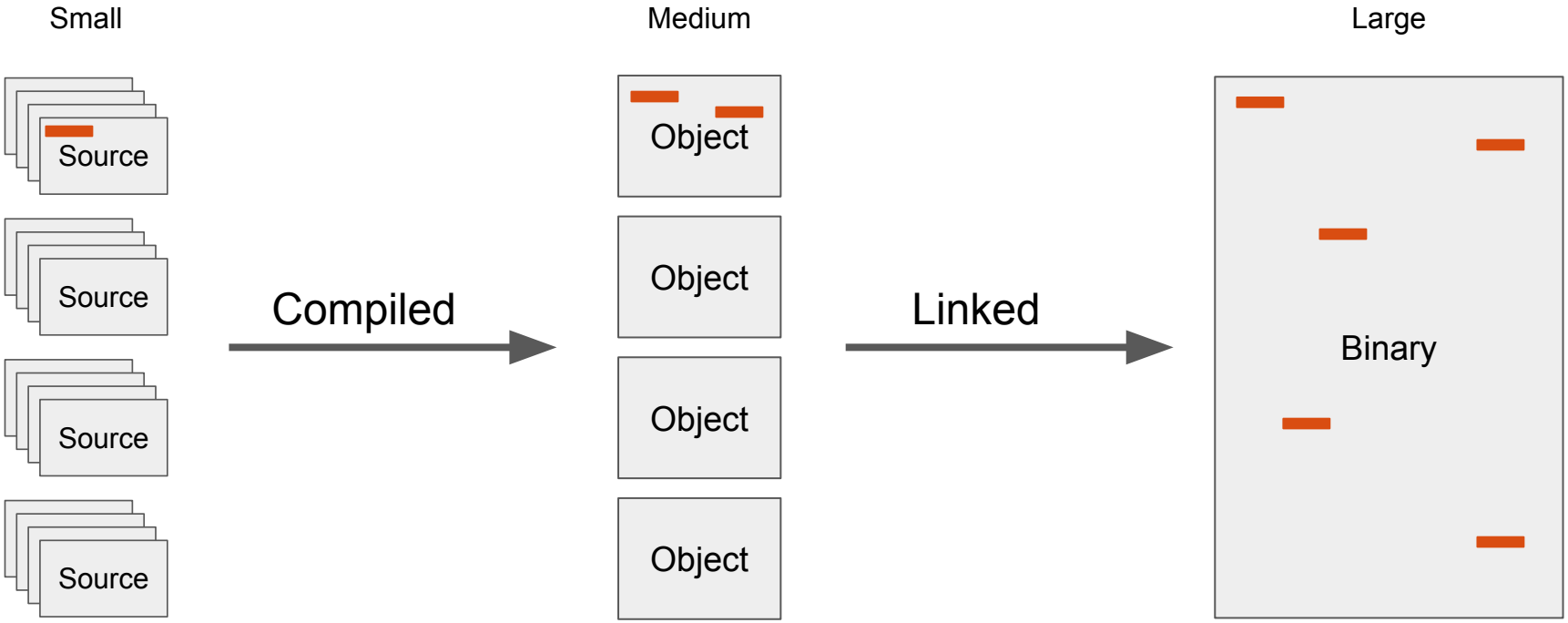
The Large Blob Problem



The Large Blob Problem



The Large Blob Problem

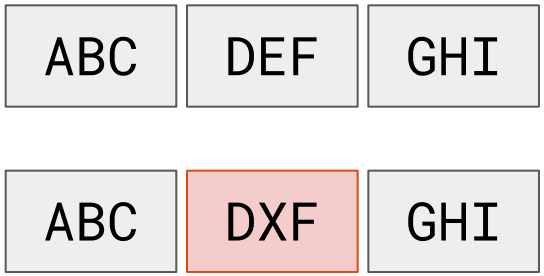




Content Defined Chunking

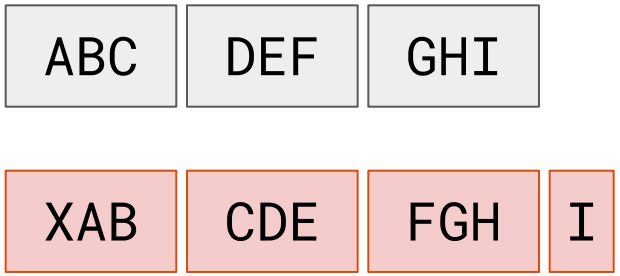
Individual bytes get mutated in place

Fixed-size chunks

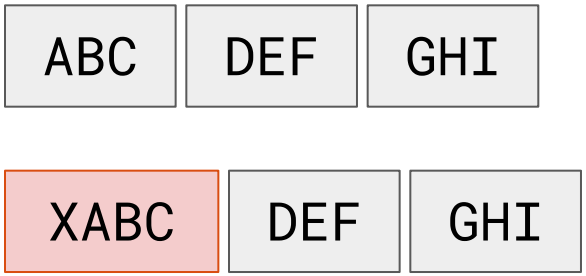


Bytes may be inserted or removed

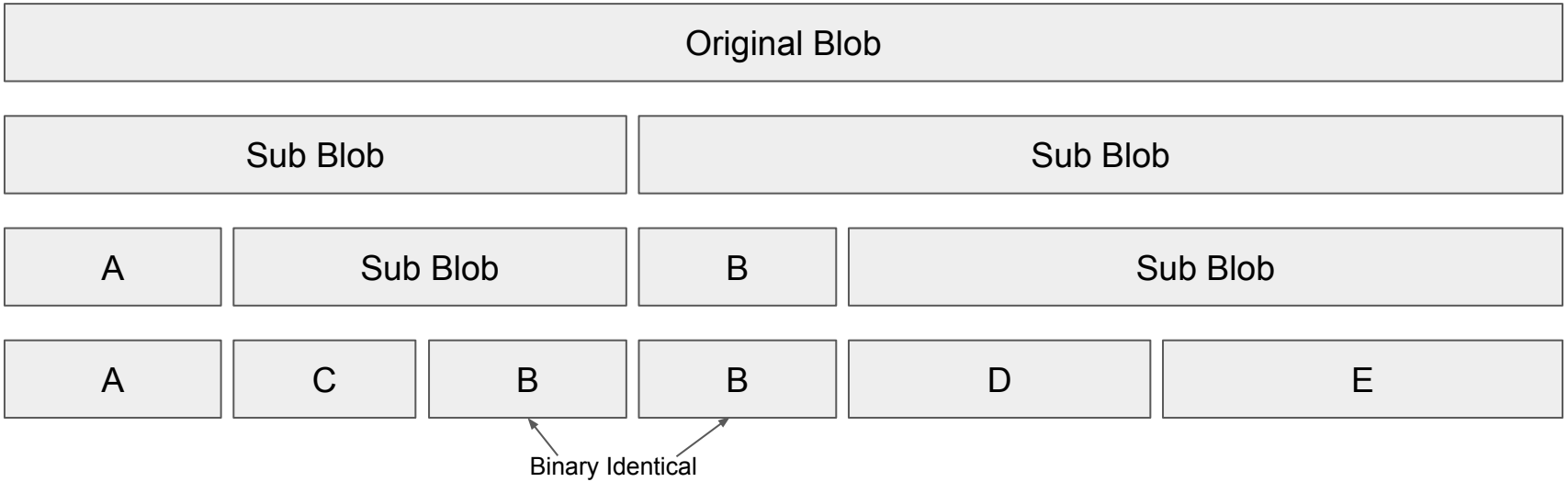
Fixed-size chunks



Content-defined chunks



RecMaxCDC



The original blob is composed of the chunks A, C, B, B, D, E

RepMaxCDC

- Any local maxima which is the largest maxima in an N*2 bytes region must be a cutting point
  - Bytes beyond such a point can not impact cutting points before that point
- The further the distance, the less likely bytes are to impact earlier cutting points

Define a maximum horizon H, beyond which we do not need to take into consideration values.

Reasonable value for H = 8N

Strong properties:

- Small parameter set (N and H)
- Streamable
- Tight Chunk Bounds
- Cheap to calculate
- Parallelizable
- Unambiguous

Changes to Buildbarn

Bimodal Distribution



of all blobs, *by count,*
are small



of all blobs, *by size,*
are big

The distribution, blobs which are
important for performance, is bimodal

New in the REv2 API

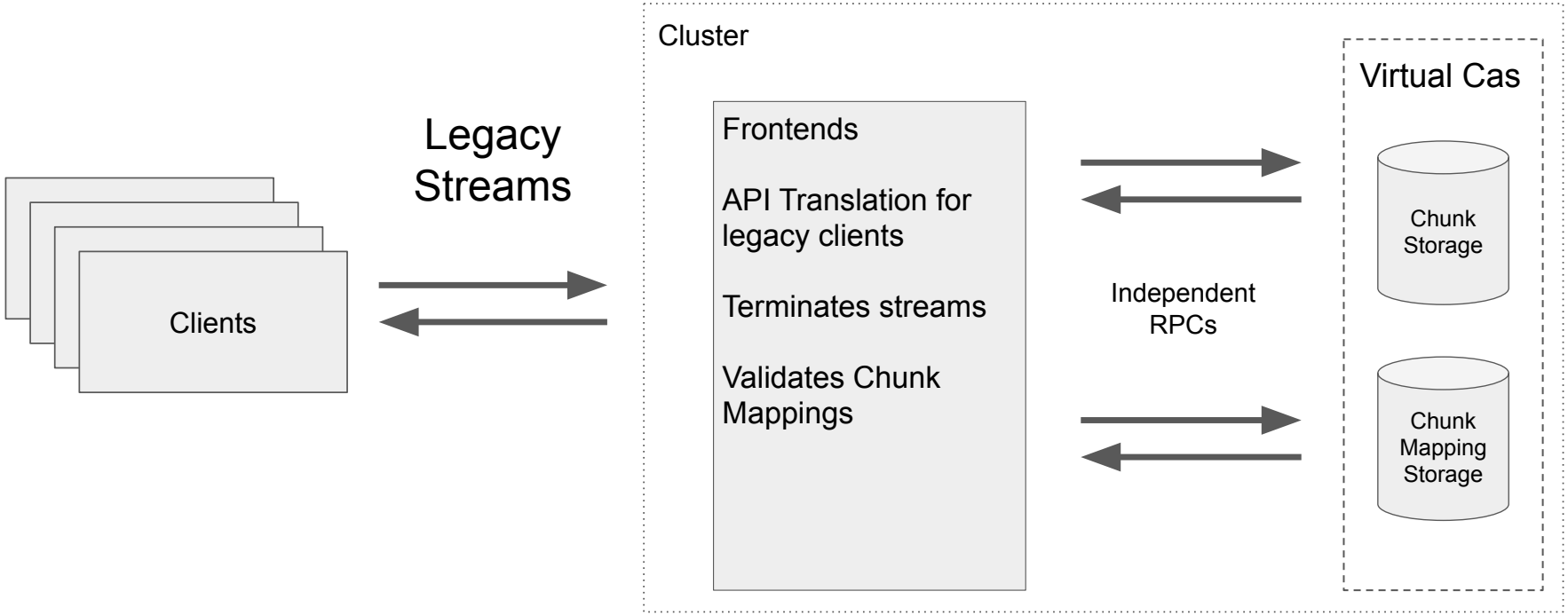
- Add support for capability discovery of CDC
- Add GetChunkMapping
- Add RegisterChunkMapping

BlobAccess Interface

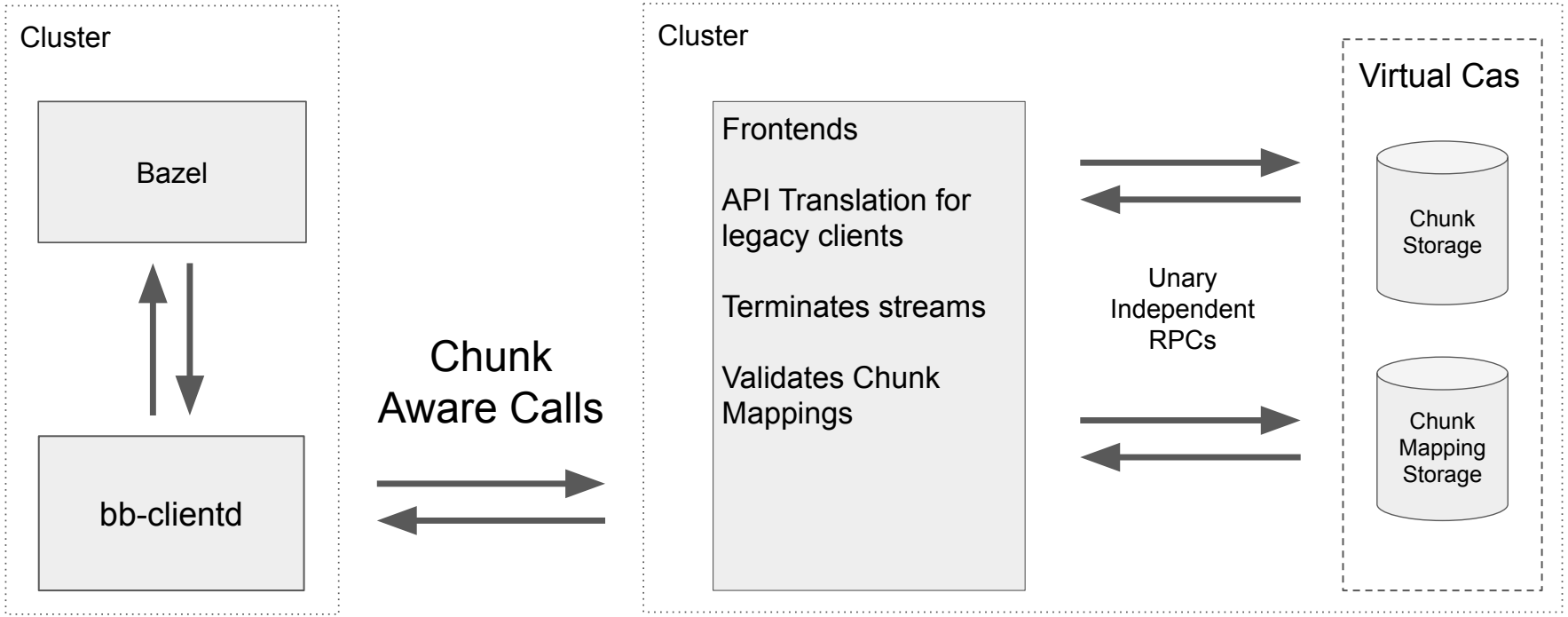
```
type BlobAccess interface {
  Get(context.Context, digest.Digest) buffer.Buffer
  Put(context.Context, digest.Digest, buffer.Buffer) error
}
```

```
type BlobAccess[T any] interface {
  Get(context.Context, digest.Digest) (T, error)
  Put(context.Context, digest.Digest, T) error
}
```

Backwards Compatibility

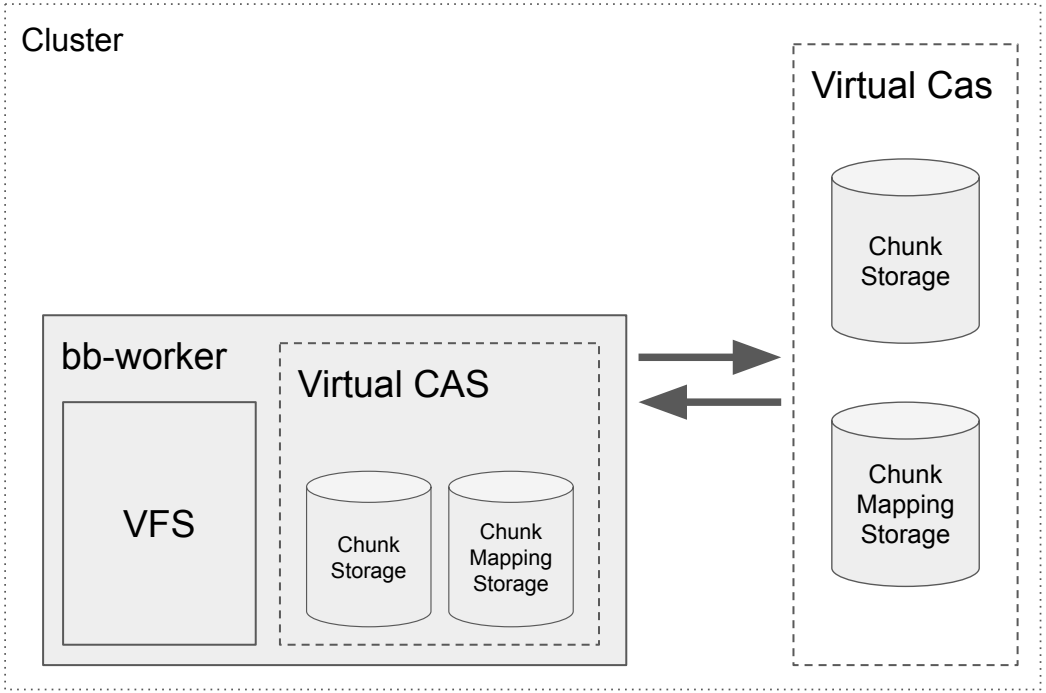


The Large Blob Problem



Results and Measurements

bb-worker - Also an REv2 Client



bb-worker - Small File Performance

Small File Performance

Open and read through 10k files of varying sizes, all small

No measurable performance difference

bb-worker - Large File Performance

Tested with two types of input files

- Repetitive File
- Random File

For two different types of access patterns

- Sequential IO
- Random IO

	Repetitive File	Random File
Random IO		
Sequential IO		

bb-worker - Large File Performance

Tested with two types of input files

- Repetitive File
- Random File

For two different types of access patterns

- Sequential IO
- Random IO

	Repetitive File	Random File
Random IO	±0%	±0%
Sequential IO		

bb-worker - Large File Performance

Tested with two types of input files

- Repetitive File
- Random File

For two different types of access patterns

- Sequential IO
- Random IO

	Repetitive File	Random File
Random IO	±0%	±0%
Sequential IO	+400%	+200%

Kernel Read Ahead

Client calling legacy api:s

Using the legacy api to upload

Measure impact of the api translation layer

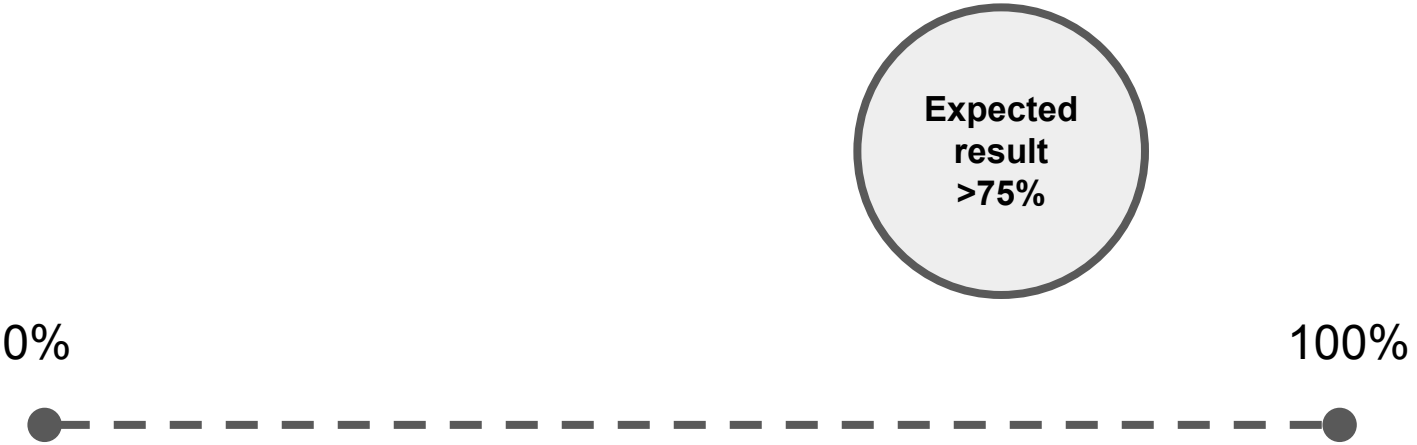
Custom client written in python to measure this pipeline

Small Blobs	±0%
Large Blobs	-50%

Impact on your storage



Impact on your storage



Most clusters should have an effective 4x more storage